

Introduction to Docker Storage



- Storage Drivers
- Volume Drivers

Storage Drivers: Where docker stores data and how it manages the filesystem of the containers

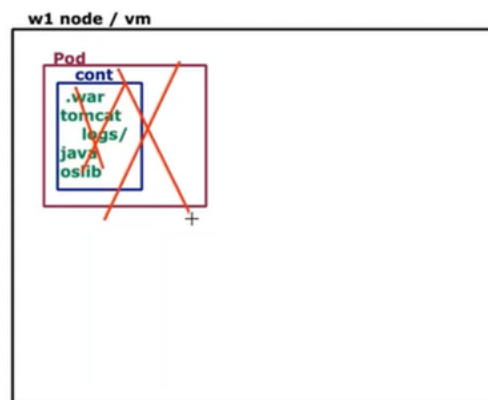
Where docker stores the data in local file system?

File System



What will happen if container layer is removed?

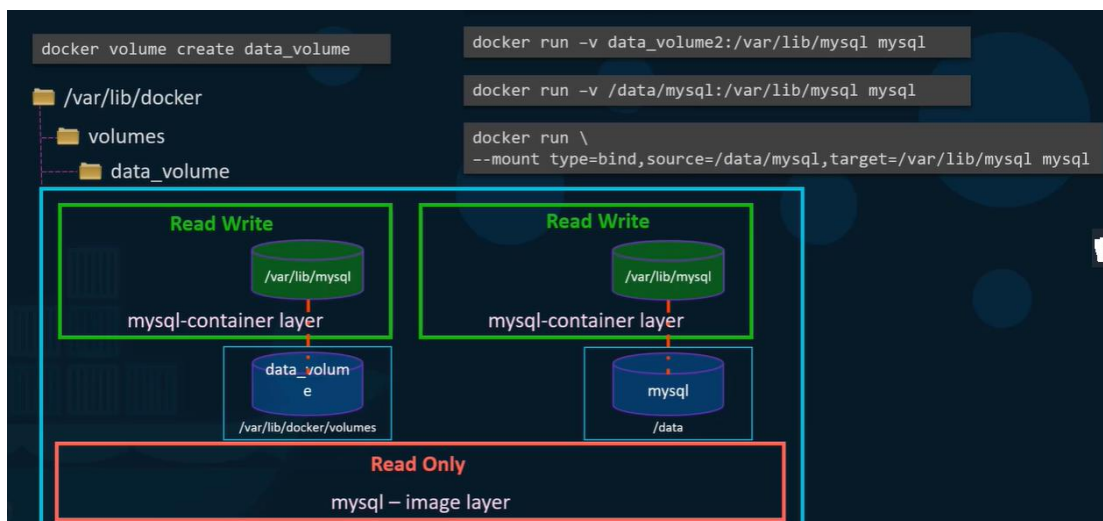
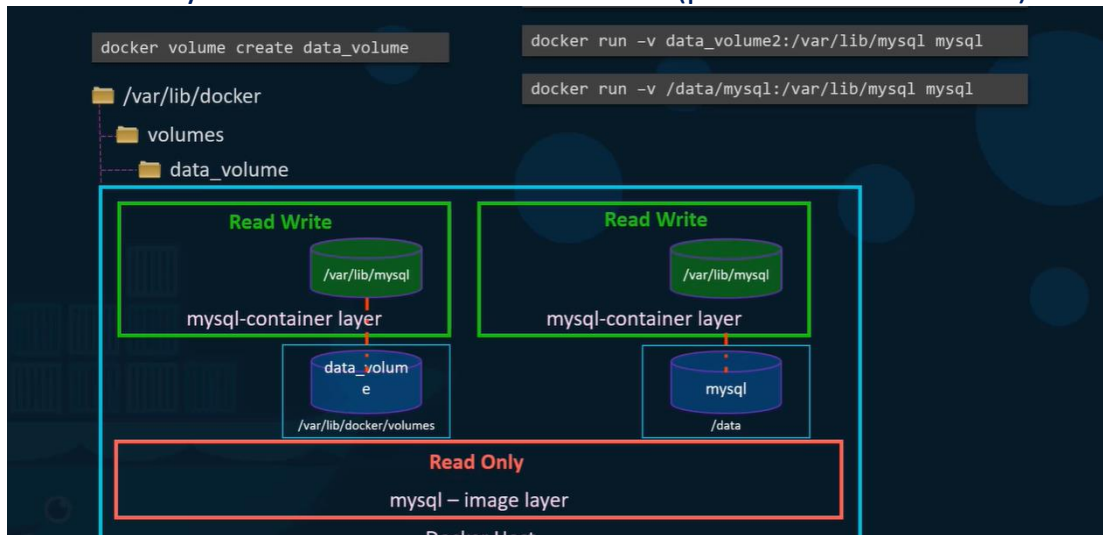
All the changes will be lost.



How we can persist the data?

- volume mount (volume created by docker)
- bind mount (mounts the data from some other location on the docker host):

If volume already exist in some another location (path of folder on host)



Storage Drivers: (Layered Architecture)

- AUFS
- ZFS
- BTRFS
- Device Mapper
- Overlay
- Overlay2

Which storage driver?

It depends on the underlying O/S e.g., in ubuntu default driver is AUFS, based on the O/S docker automatically choose the best driver.

Volume driver plugin in docker

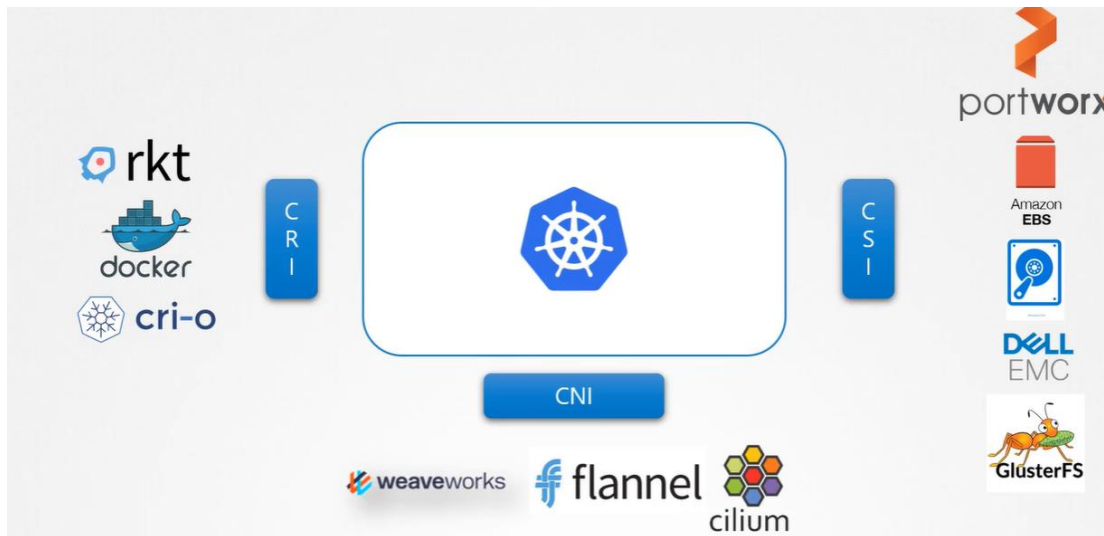
If you want persist storage, you must create volumes, remember volumes are not handled by storage drivers, volumes are handled by volume driver plugin, and default volume driver is local store volume/**var/lib /volume**

STORAGE DRIVERS	VOLUME DRIVERS
AUFS ZFS BTRFS DEVICE MAPPER OVERLAY	Local Azure File Storage Convoy DigitalOcean Block Storage Flocker gce-docker GlusterFS NetApp RexRay Portworx VMware vSphere Storage

Container Run Time Interface (CRI): How container runtime communicates with their orchestration (K8s)

Container Network Interface (CNI): Support different network solutions

Container Storage Interface (CSI): Support different storage solutions

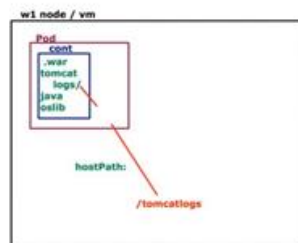


CSI -is not a K8s standard it's a universal standard
Storage driver should implement the specification.

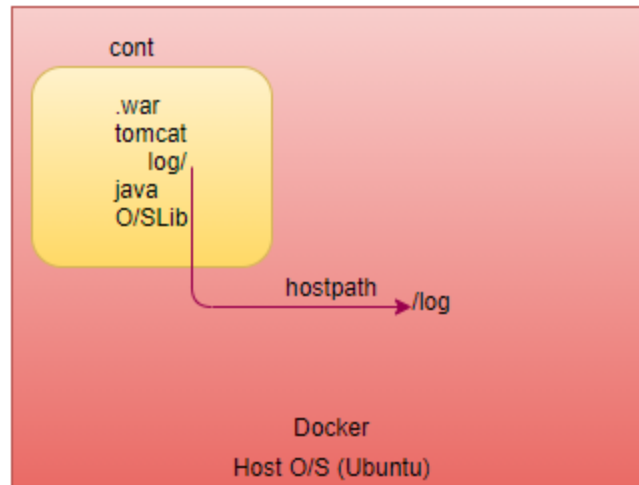
Volumes:

Kubernetes Volumes
PersistentVolume
hostPath/NFS
PersistentVolumeClaim

— pv can not be accessed by any pod directly in kube
— pod has to use "persistent volume claim" to access a PV in kube
+



docker volume- making the data persistent VM/EC2



Storage-In K8s

Use Case: At some point, your apps require storage where the data is stored and accessed.

- **How** does this storage volumes are handled inside the kubernetes cluster?
- **How** does data persist beyond pod life?
- **How** can containers share files between containers in Pod?

UC1: All the data generated by container *resides inside the container*, e.g., logs generated by tomcat server, **what will be happen if we lose the container?**

All data will be lost, data generated by container is temporary.

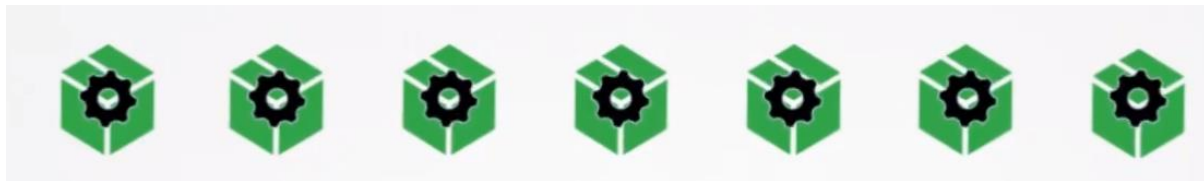
In case of application failed, what will you validate? Logs of the application, but where is the logs? **inside the container**, Process running inside container always generate the data inside the container, how we make data persistent

What happen if we lose the container?

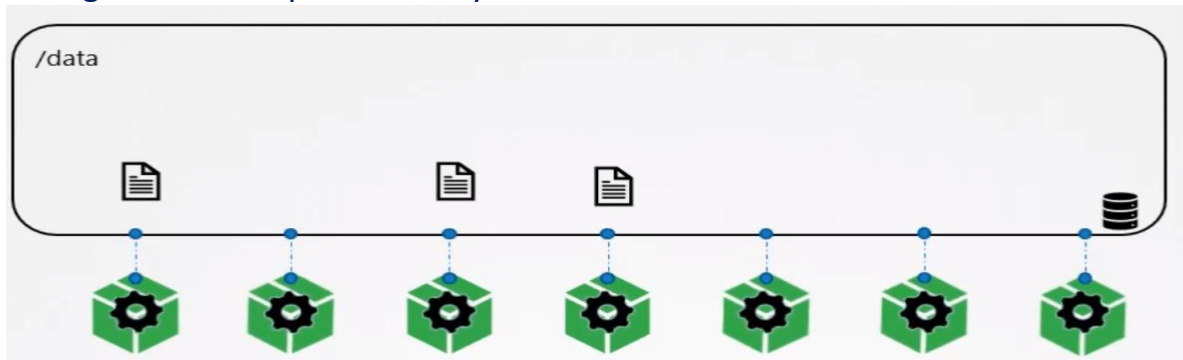
Sol:

What happen if we lose the Pod? **Pod is also temporary**. If we lose the pod then can't get back.

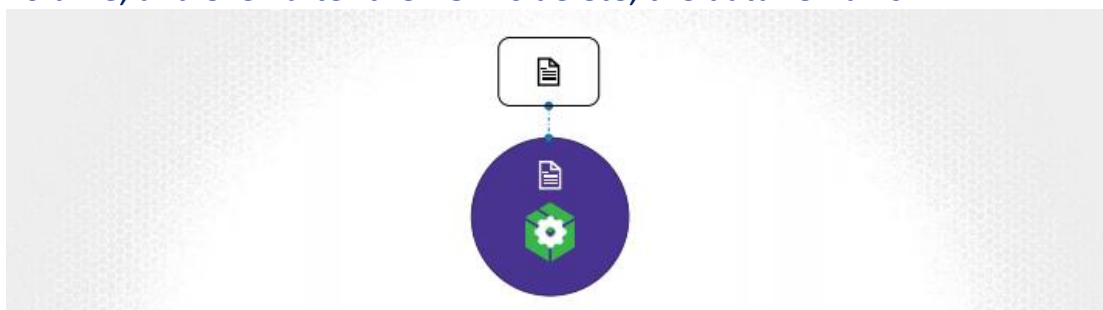
Let us look at volumes in Docker first. **Docker containers are meant to be transient in nature. Which means they are meant to last only for a short period of time.** They are called upon when required to process data and destroyed once finished. The same is true for the data within the container. The data is destroyed along with the container.



To persist data processed by the containers, we attach a volume to the containers when they are created. The data processed by the container is now placed in this volume, thereby retaining it permanently. Even if the container is deleted, the data generated or processed by it remains.



So how does that work in the Kubernetes world. Just as in Docker, the PODs created in Kubernetes are transient in nature. When a POD is created to process data and then deleted, the data processed by it gets deleted as well. For this we attach a volume to the POD. The data generated by the POD is now stored in the volume, and even after the POD is deleted, the data remains.

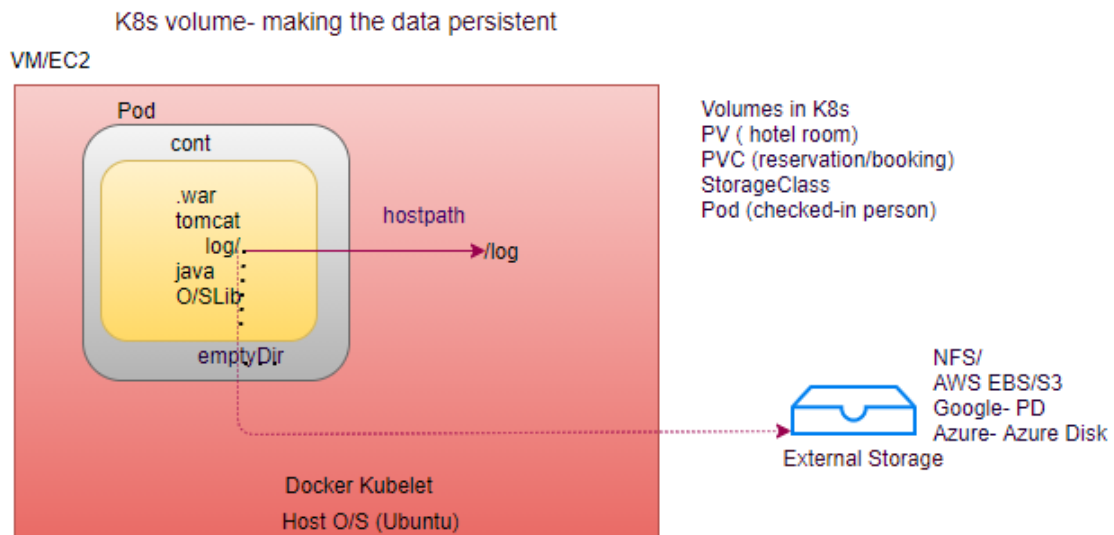


Let's look at a simple implementation of volumes. We have a single node Kubernetes cluster. We create a simple POD that generates a random number between 1 and 100 and writes that to a file at `/data/number`. Out and then gets deleted along with the random number. **To retain the number generated by the pod, we create a volume and a volume needs a storage.** When you create a volume, you can choose to configure its storage in different ways.

How to access volume from container?

Once the volume is created, to access it from a container **we mount the volume to a directory inside the container. We use the `volumeMounts` field in each**

container to mount the data-volume to the directory /opt within the container. The random number will now be written to **/opt mount inside the container**, which happens to be on the data-volume.

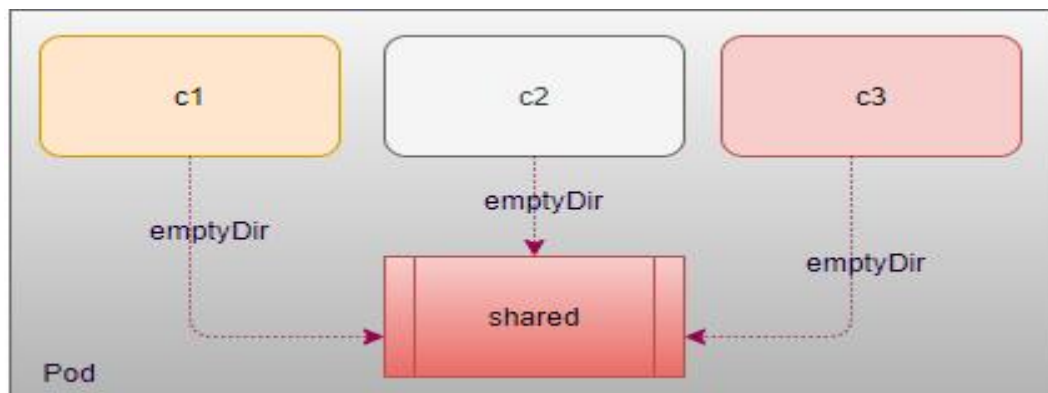


Volume Storage option:

- emptyDir
- hostPath
- nfs
- PV
- PVC

emptyDir:

UC: We have 3 containers running inside the Pod we want to share data between all the three containers.



emptyDir is just like a shared drive within the Pod.

kubectl exec -it <pod name> -c <container name> -- bash
df -h

volumes:

- **name:**
emptyDir: {}

- Creates empty directory (cache) first created when a pod is assigned to a Node.
- Stays as long as pod is running.
- Once pod is removed from a node, emptyDir is deleted forever.

Use Case

Temporary Space

Caching

Limitation: If we delete the Pod, shared directory also deleted.

hostPath:

Mounts a file or directory from the host node's filesystem into your Pod

Remains even after the pod is terminated similar to docker volume.

Lifecycle till host.

volumes:

- **name:**
hostPath:
 path: # If path not exist, then first create a folder in host node
 # this is the path on the worker node where the pod will run
 type:

Limitation: shared data between host and Pod, in case of Pod rescheduled on other node what about the data on previous node? Will be lost.

nfs:

Shared drive, just like google drive (google disk, aws disk)

volumes:

- **name:**
nfs:
 path:
 server:

Cloud's Solutions:

GCP

Home-> Kubernetes Engine->Clusters

Home-> Network Service-> LoadBalancer

Home->Compute Engine-> disk

gcePersistent Disk

Volume mounts a Google Compute Engine (GCE) Persistent Disk into Pod.

Restrictions

You must create a PD using gcloud or the GCE API or UI before you can use it.

The nodes on which pods are running must be GCE VMs.

Those VMs need to be in the same GCE project and zone as the PD (persistent disk).

volumes:

- **name:**
gcePersistentDisk:
pdName: # disk name in google cloud
fsType: ext4

pdName: persistent name (disk name in Google cloud)

fsType: raw disk formatted by ext4

awsElasticBlockStore:

Volume mounts with awsElasticBlockStore into Pod.

volumes:

- **name:**
awsElasticBlockStore:
volumeID: # volume id of EBS
fsType: ext4

How traditional Storage request process works?

You have storage admin creates a chunks of storage pieces which are called as LUNS, then developer requests these storage LUNS to mount on the operating systems, after that developer team stores application data on these storage LUNS.

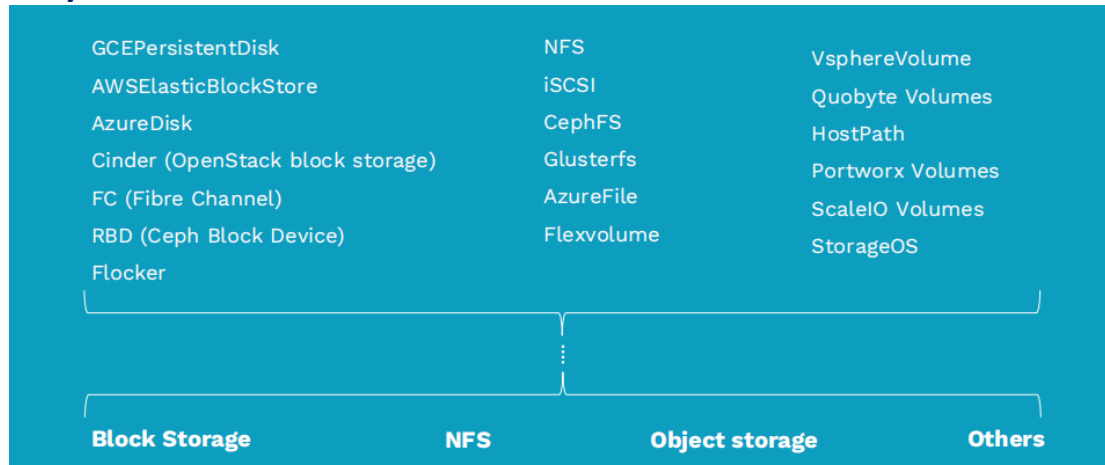
Your company's infrastructure team provides different storage solutions such as Block, NAS, Object storage, AWS disks and more.

There need to be a **consistent way to deal** with all these storage types.

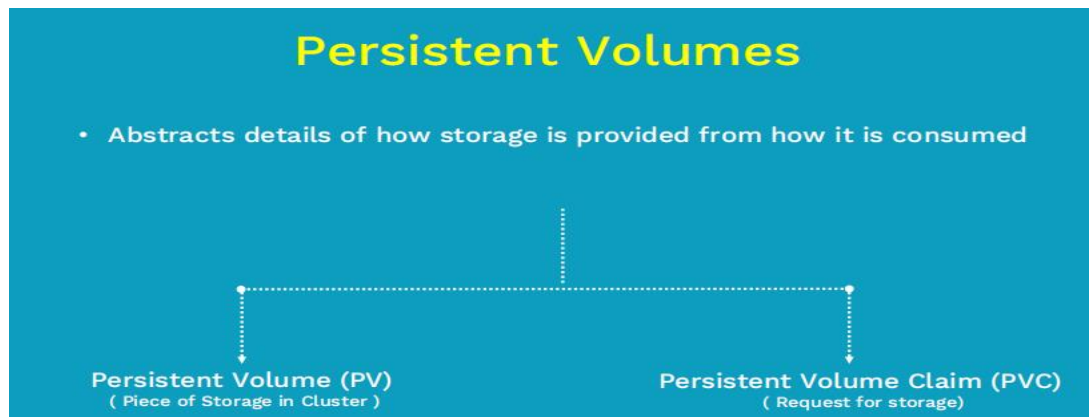
What is kubernetes solution for that?

PV and PVC

Why we need PV?



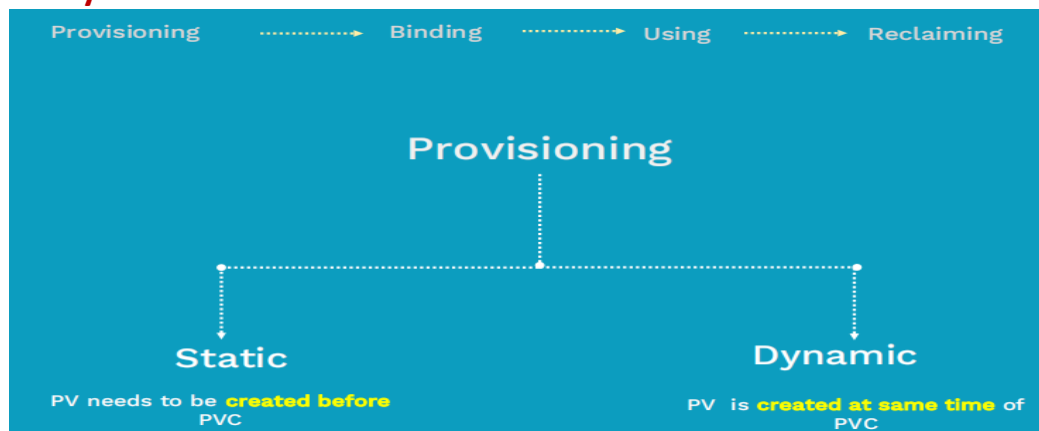
There are different storage infrastructures are available, use these different storage type for different purposes based on the data we are storing on it and the way we are access it, these storage types typically fall under Block, NFS, Object and Other storage type categories. So, each of them has different architecture and different APIs, problem here is if we want to attach and manage these driver storage type manually, you have to develop customized plugins to interact with external drives API such as mounting the disk requesting the capacity and managing the lifecycle of the disk and etc.so we have to also configure cloud storage environment. So all of which result in unnecessarily overhead so we need one storage interface for all durable storage options and k8s solution for this is PersistentVolumes. Kubernetes platform simplifies storage management for you PersistentVolumes subsystem (PV and PVC) provides a standard API for developers and administrator that abstracts details of how storage is provided from and how it is consumed.so the it will let Storage administrator and developers to focus on capacity and storage types their application is consumed rather than each and every detail of storage provider API.



PV: Piece of pre-provisioned storage in cluster, provisioned by administrator. Data inside this storage exist beyond the life cycle of Pod.

PVC: is a Storage request by developer. So, developer request for some capacity along with some access-mode.

Lifecycle of PersistentVolume



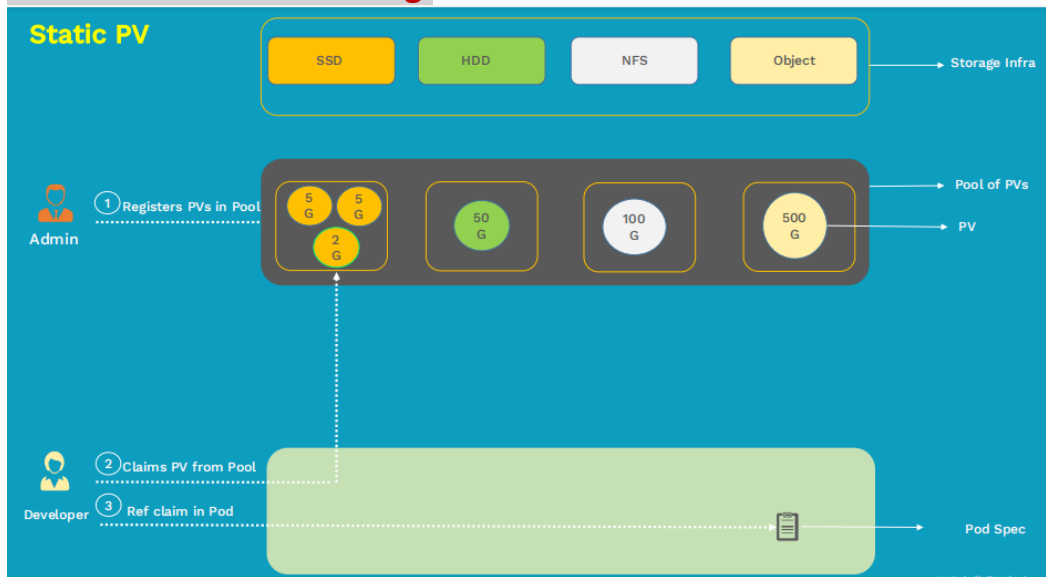
Provisioning: administrator creates a storage volume; these volumes can be any storage types. In K8s terminology **these volumes are called PV**.

Once Storage is provisioned next step is binding

Binding: In this stage we bind the storage request to the PV that was provisioned earlier stage. **This storage request in K8S terms is called as PVC.** Typically a developer creates this PVC to request specific amount of storage and access-mode.

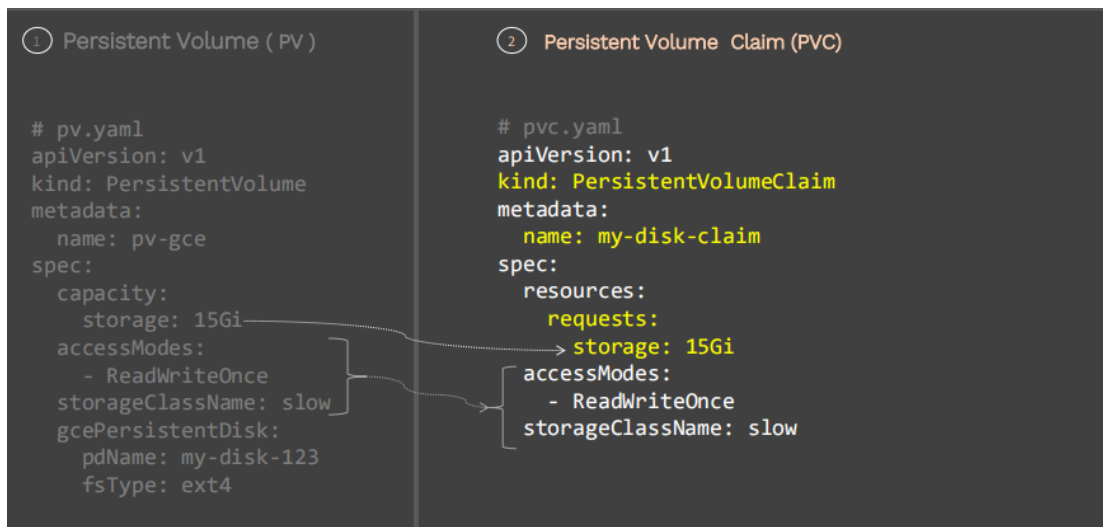
As soon as PVC request is created a control loop on k8s master watches the new PVC request and binds the matching PV if it is available in the cluster.

Static Volume Provisioning:



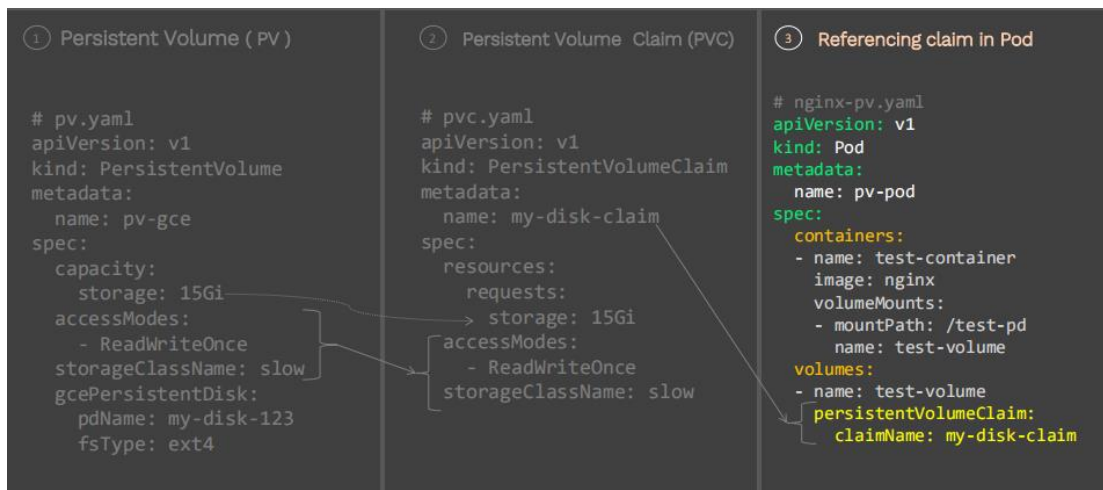
Steps:

- Create a PersistentVolume
- Create a PersistentVolumeClaim
- Refereeing Claim in Pod
- Test Use Case



Reference Persistent Volume Claim inside the Pod

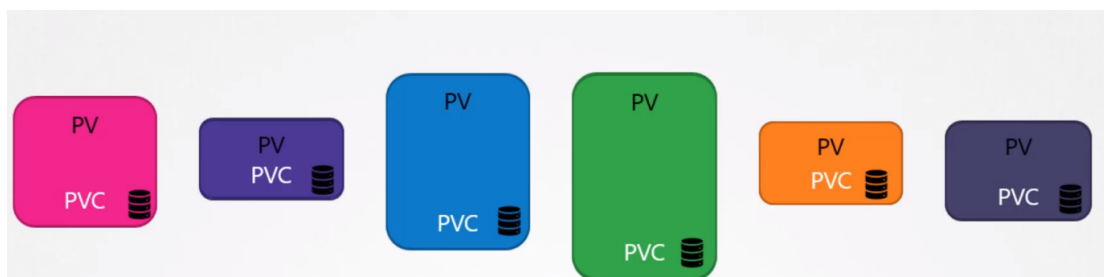
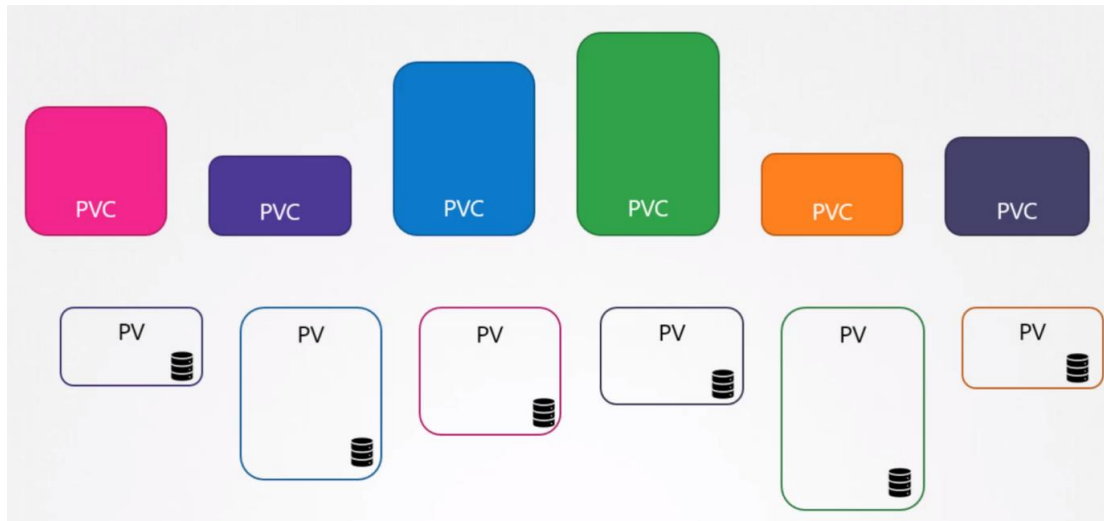
To claim the Volume inside the Pod we have to define the Persistent Volume Claim



What if matching PV is not available?

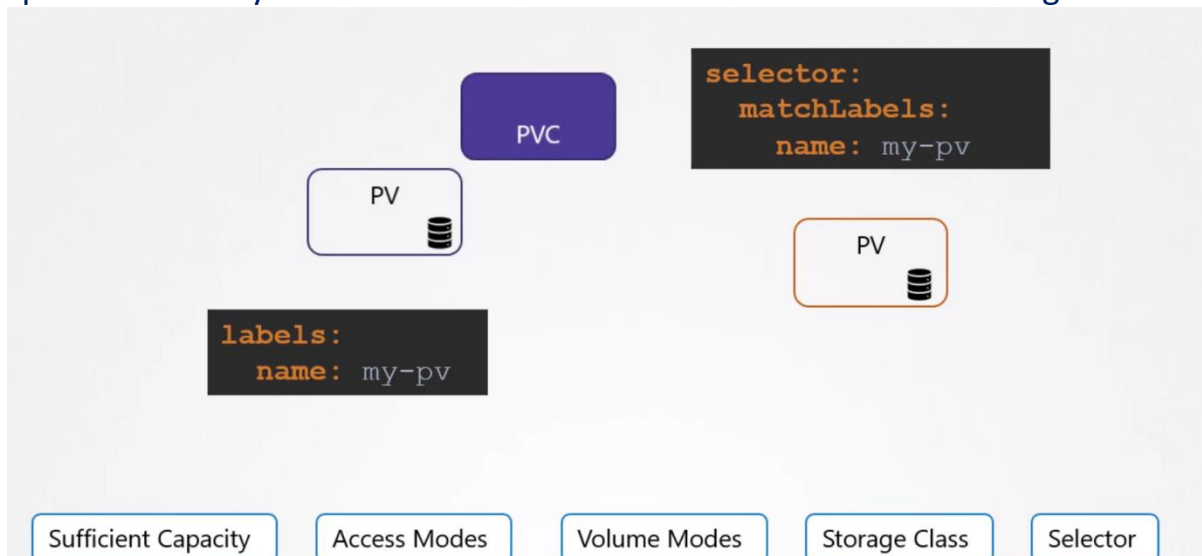
Case 1: requested for 100Gi memory but in available memory chunks in pool is only 50Gi, in that case PVC request has to wait until 100Gi chunk storage created and added in to the pool. To solve this issue dynamic PV came in picture.

Case 2: requested for 10Gi memory but in available memory chunks in pool is only 12Gi, in this case PVC request bound to the 12Gi memory chunk because the user request at least for 10Gi. but the excess must not be so much. In binding stage K8S binds user request with available storage in the pool. Once binding is done next stage, developer start using this storage

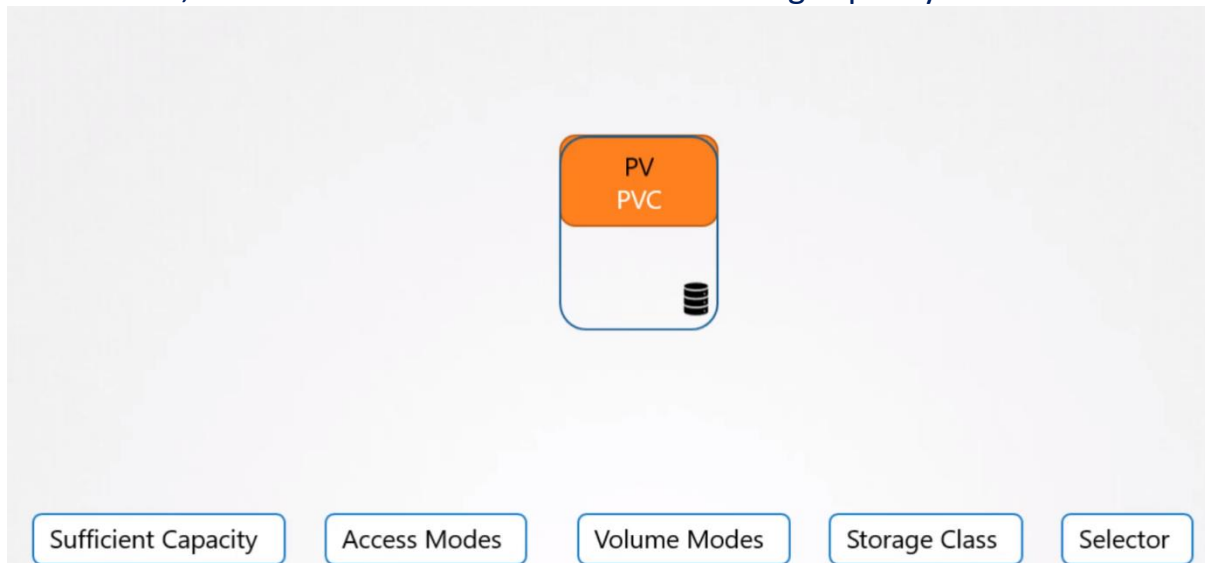


Every PVC bound to the single PV, during binding process K8s tries to find out the PV that has sufficient capacity as requested by the claim.

If there are multiple options are available for single claim and you want to use specific volume you can still use labels and selectors to bind to the right volume.



It may be possible that smaller claim bound to the larger volume if all other criteria match, and there are no other options there is 1:1 relationship between PV and PVC, so no other claim can use the remaining capacity of volume.



If there is no PV available the PVC will remain in pending state until newer Volume made available to the cluster, once newer Volume is available, claim will automatically to be bound to the newly available volume.



pvc-definition.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 500Mi
```

pv-definition.yaml

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-vol1
spec:
  accessModes:
    - ReadWriteOnce
  capacity:
    storage: 1Gi
  awsElasticBlockStore:
    volumeID: <volume-id>
    fsType: ext4
```

How to take backup of PV?

PV is at cluster level, k8s doesn't provide backup of PV, and you have to write your own script for backup purpose.

accessModes:

ReadWriteOnce: Only one node can read/write the data to the volume.

ReadWriteMany: one node can read/write the data to the volume.

ReadOnlyMany:

RECLAIM POLICY:

When user is done with their volume, they can delete the PVC, from the K8S which allows the reclaiming of the resources.

Recycle: Delete the data when its release in PVC (deleted), and can be used by other PVC.

Retain: Data will be retained even after released the PVC (deleted), can't be used by other PVC.

Delete: PV also deleted.

/*

ReadWriteOnce- The volume can be mounted as read-write by a single node (RWO): access by only pods running on same node.

ReadOnlyMany- The volume can be mounted as read-only by a many node (ROX): all pods running on any node can read data from volume.

ReadWriteMany- The volume can be mounted as read-write by a many node (RWX): access by any pods running on any node.

Recycle- If you delete the PVC, data will be erased and can be assigned to others (first delete pod, only after that PVC can be deleted)

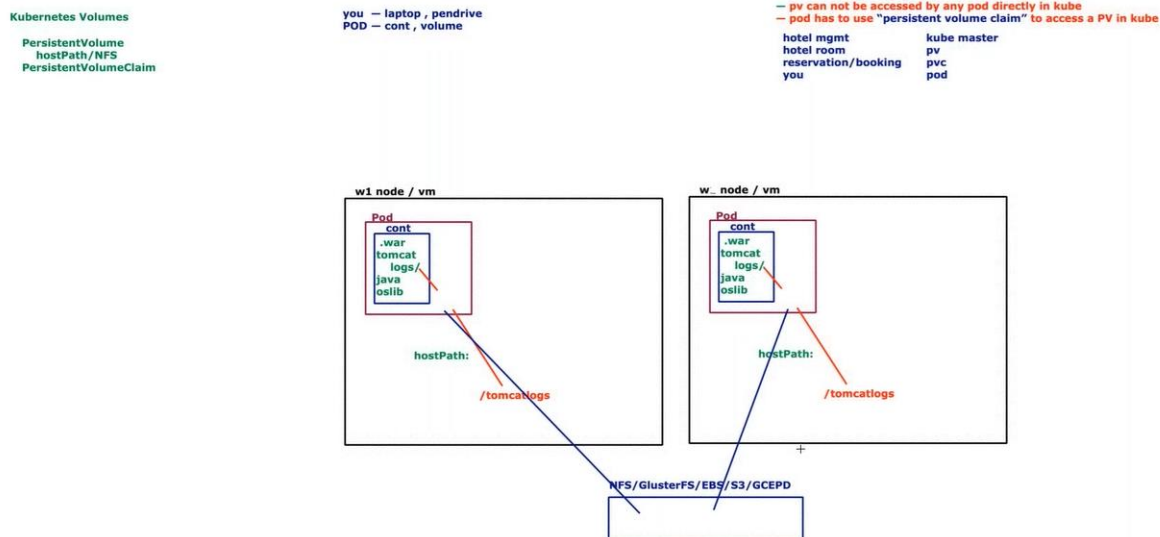
Retain- If you delete the PVC, PV still can't be assigned to others. (First delete pod, only after that PVC can be deleted)

Delete- If you delete the PVC, data will be erased and PV also deleted (first delete pod, only after that PVC can be deleted)

*/

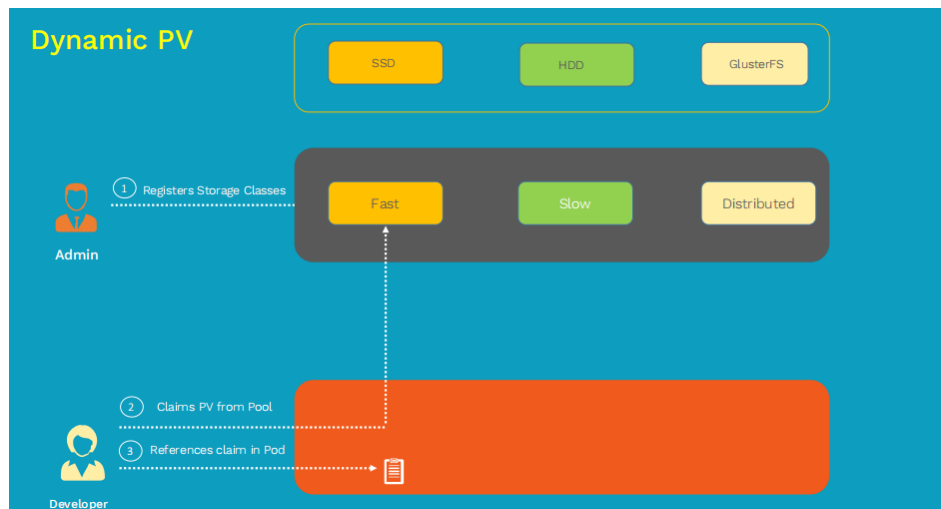
How to write Logs?

Log can be segregated to the external to the application ELK or Splunk



Dynamic Volume Provisioning:

Advantage is don't need to create a PersistentVolume in advance, Instead we create a **StorageClass**, This storage classes are generally a tag given to a storage based on the type of medium in the backend, e.g. fast storage class for all SSD drive and low storage type for HDD drive created by administrator, also administrator has to define default storage type e.g. define slow as a default storage type, If developer don't define storage class in PVC then default storage class assigned to developer. Internally it creates a PV and bound to the PVC.



In case of Dynamic provisioning developer has not worry about the storage available or not.

StorageClass (Dynamic Provision):

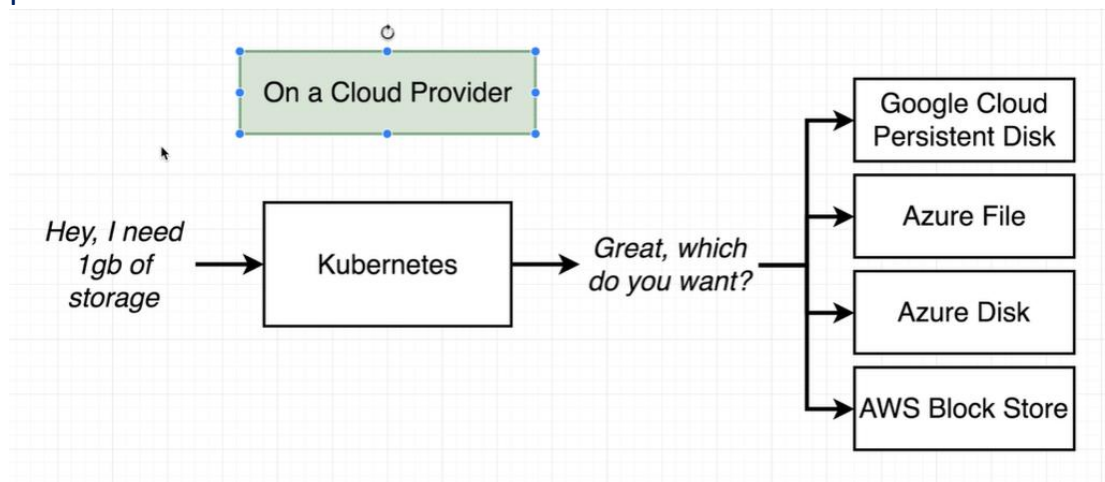
StorageClass: Storage class **in cloud** allow the provision of increase memory dynamically, dynamically provision the disk, which is share among all nodes, cloud provider provides block storage in their cluster level

kubectl get storage # don't need to create PV, created by cloud provider

GCP->compute engine -> disks

We don't want to create PV beforehand.

Use Case: In case of static provisioning, if we create pv first means we have to preserve storage and we have to pay for that storage either we use it or not use, in that case StorageClass (In Cloud) help us by providing dynamic provision of PV. It will create external disk and create a pv dynamically, can be claimed through pvc.



To create a pv disk space must be available, instead of pv if we create a storage class no disk space required, this storage Class in cloud have capability to create disk space (pv) dynamically.

kubectl get sc

sc-definition.yaml

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: google-storage
provisioner: kubernetes.io/gce-pd
parameters:
  type: pd-standard
  replication-type: none
```

pv-definition.yaml

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-vol1
spec:
  accessModes:
    - ReadWriteOnce
  capacity:
    storage: 500Mi
  gcePersistentDisk:
    pdName: pd-disk
    fsType: ext4
```

sc-definition.yaml

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: google-storage
provisioner: kubernetes.io/gce-pd
```

PV

SC

① Storage Class

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: fast
provisioner: kubernetes.io/gce-pd
parameters:
  type: pd-ssd
```

② Persistent Volume Claim (PVC)

```
# pvc-dv1.yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: my-disk-claim-1
spec:
  resources:
    requests:
      storage: 30Gi
  accessModes:
    - ReadWriteOnce
  storageClassName: fast
```

```
# pvc-dv2.yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: my-disk-claim-2
spec:
  resources:
    requests:
      storage: 40Gi
  accessModes:
    - ReadWriteOnce
  storageClassName: fast
```

